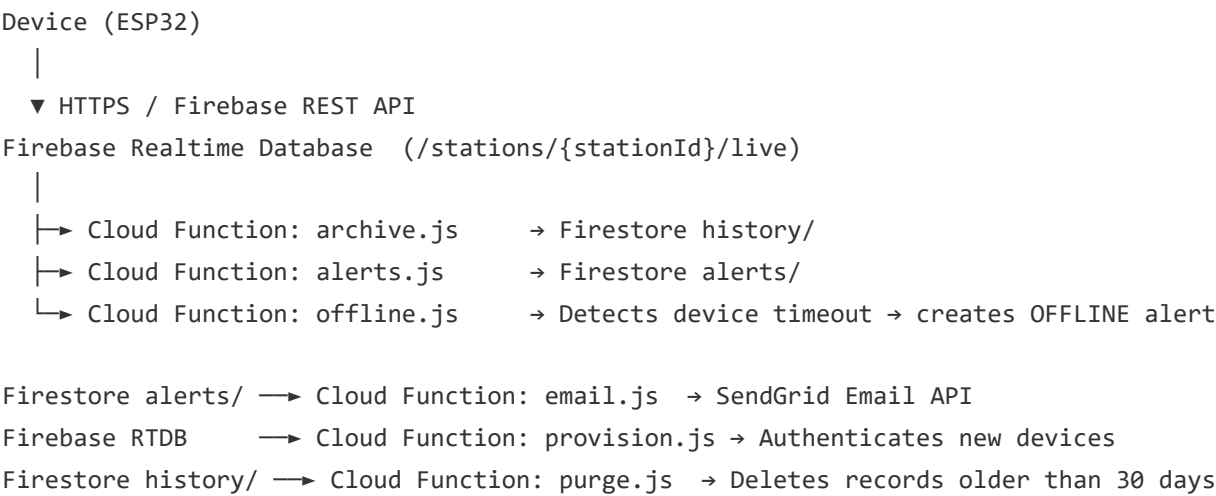


5 — Backend & Cloud Setup Guide

Audience: Developer / DevOps **Stack:** Google Firebase — Realtime Database · Firestore · Cloud Functions (Node.js 18) · Firebase Authentication · Firebase Storage

Architecture Summary



Prerequisites

Tool	Version	Install
Node.js	18.x or 20.x	nodejs.org
Firebase CLI	Latest	<code>npm install -g firebase-tools</code>
Google Account	—	With a Firebase project created

Step 1 — Firebase Project Setup

- Go to [Firebase Console](#) and create a new project named `pumping-station-iot`.
- Enable the following services in the Firebase Console:

Authentication

- Go to **Authentication** → **Sign-in method**

- Enable **Email/Password**

Realtime Database

- Go to **Realtime Database** → **Create database**
- Select **Europe West** region
- Start in **locked mode** (rules are deployed separately)

Firestore

- Go to **Firestore Database** → **Create database**
- Select **Europe** location
- Start in **production mode**

Storage

- Go to **Storage** → **Get started**
- Accept default rules (will be overridden on deploy)

Cloud Functions

- Go to **Functions** → **Get started**
- Select **Node.js 18** runtime
- Upgrade to **Blaze plan** (pay-as-you-go — required for Functions outbound network calls)

Step 2 — Configure Environment Secrets

Cloud Functions use environment secrets for the email service (SendGrid):

```
cd backend
firebase functions:secrets:set SENDGRID_API_KEY
# Paste your SendGrid API key when prompted
```

Also set the sender email:

```
firebase functions:secrets:set SENDGRID_FROM_EMAIL
# e.g. alerts@yourdomain.com
```

Step 3 — Configure the Frontend Firebase Settings

In the Firebase Console:

1. Go to **Project Settings** → **General** → **Your apps**
2. Click **Add app** → **Web app**
3. Copy the config object

Open `frontend/.env.example`, copy it to `frontend/.env`, and fill in your values:

```
cp frontend/.env.example frontend/.env
```

Edit `frontend/.env`:

```
VITE_FIREBASE_API_KEY=your_api_key
VITE_FIREBASE_AUTH_DOMAIN=your-project.firebaseio.com
VITE_FIREBASE_DATABASE_URL=https://your-project-default-rtdb.europe-west1.firebaseio.com
VITE_FIREBASE_PROJECT_ID=your-project-id
VITE_FIREBASE_STORAGE_BUCKET=your-project.firebaseio.com
VITE_FIREBASE_MESSAGING_SENDER_ID=your_sender_id
VITE_FIREBASE_APP_ID=your_app_id
VITE_FIREBASE_MEASUREMENT_ID=your_measurement_id
```

Note: The frontend source code reads these values automatically via `import.meta.env`. You do **not** need to edit any TypeScript files.

Also update `firmware/src/config.h`:

```
#define FIREBASE_PROJECT_ID "your-project-id"
#define FIREBASE_API_KEY    "YOUR_WEB_API_KEY"
#define FIREBASE_DB_URL     "https://your-project-default-rtdb.europe-west1.firebaseio.com"
```

Step 4 — Deploy Backend

```
cd backend
```

```
# Login to Firebase
firebase login
```

```
# Install Cloud Functions dependencies
cd functions && npm install && cd ..
```

```
# Deploy everything (rules + functions)
firebase deploy
```

```
# Or deploy individual parts:
```

```
firebase deploy --only functions
firebase deploy --only firestore:rules
firebase deploy --only database:rules
firebase deploy --only storage
```

Expected Output

```
✓ functions[archive]: nodejs18 function initialized.
✓ functions[alerts]: nodejs18 function initialized.
✓ functions[email]: nodejs18 function initialized.
✓ functions[offline]: nodejs18 function initialized.
✓ functions[provision]: nodejs18 function initialized.
✓ functions[purge]: nodejs18 function initialized.
✓ Deploy complete!
```

Step 5 — Deploy Frontend

```
cd frontend
npm install
npm run build
firebase deploy --only hosting
```

Or host on your own server by copying the contents of `frontend/dist/` to any static web host (Nginx, Apache, Vercel, Netlify, etc.).

Cloud Functions Reference

Function	Trigger	Purpose
archive	RTDB write at <code>/live/{stationId}</code>	Archives telemetry snapshot to Firestore <code>history/</code>
alerts	RTDB write at <code>/live/{stationId}</code>	Evaluates thresholds, creates/resolves alerts in Firestore
email	Firestore write to <code>alerts/</code>	Sends alert email via SendGrid
offline	RTDB delete / Pub/Sub schedule	Detects stations not reporting for >5 min, triggers OFFLINE alert
provision	HTTPS callable	Validates provisioning token, creates Firebase Auth user for device

Function	Trigger	Purpose
purge	Pub/Sub schedule (daily)	Deletes Firestore history records older than 90 days

Database Security Rules

Realtime Database (`database.rules.json`)

- Devices can only **write** to their own `/live/{stationId}` node (authenticated via device UID)
- Admins/Operators can **read** all live nodes

Firestore (`firestore.rules`)

- `alerts/` — readable by all authenticated users, writable only by Cloud Functions (service account)
- `history/` — readable by authenticated users, writable only by Cloud Functions
- `stations/` — readable by all, writable only by Admin role
- `users/` — Admin-only read/write

Monitoring & Logs

View Cloud Function logs in the Firebase Console:

- **Functions** → **Logs**

Or from CLI:

```
firebase functions:log --only archive
firebase functions:log --only alerts
```

Cost Estimation (at Scale)

Resource	Usage at 400 stations	Estimated Monthly Cost
RTDB writes	400 stations × 2 writes/min × 43,200 min = ~34M ops	~\$3.40

Resource	Usage at 400 stations	Estimated Monthly Cost
Firestore writes (history)	$400 \times 2/\text{min} = \sim 34\text{M writes}$	$\sim \$10.20$
Cloud Functions invocations	$\sim 70\text{M/month}$	$\sim \$0.70$
Storage (firmware OTA)	$\sim 100\text{MB}$	$\sim \$0.02$
Total		$\sim \\$14\text{--}20 / \text{month}$

Blaze plan billing is purely pay-per-use. Idle months cost nearly \$0.

Next: [Alerts & Scaling Guide →](#)